

Web Requests Hypertext Transfer Protocol Secure (HTTPS)

2.02.1. Проблема HTTP

HTTP передаёт все данные в **незашифрованном виде (clear-text)**. Это позволяет злоумышленнику между источником и получателем выполнить **атаку «человек посередине» (MITM)** и перехватить чувствительную информацию.

Пример: в перехваченном HTTP-запросе к `/login.php` видны логин и пароль в открытом виде.

Чтобы устранить эту уязвимость, был разработан протокол **HTTPS (HTTP Secure)**, в котором все данные передаются в **зашифрованном** виде. Даже при перехвате трафика злоумышленник не сможет извлечь полезную информацию.

- **HTTPS** становится стандартом де-факто для большинства сайтов.
- **HTTP** постепенно вытесняется, браузеры скоро перестанут поддерживать его по умолчанию.

2.02.2. Обзор HTTPS

При использовании **HTTPS** весь обмен между клиентом и сервером шифруется (например, с помощью **TLS**). Это делает перехваченные данные нечитаемыми — в анализаторе трафика (Wireshark) мы увидим только поток зашифрованных байт.

Протокол	Особенность
HTTP	Данные передаются открыто, уязвим для MITM
HTTPS	Данные шифруются (TLS/SSL), защита от перехвата

Table 1: Сравнение HTTP и HTTPS

Признаки использования **HTTPS**:

- В URL — схема `https://` (например, `https://www.google.com`).
- В адресной строке браузера — значок **замка** (слева от URL).

Важно: даже при HTTPS запрос может раскрыть посещаемый URL через **DNS-запросы** (если они не зашифрованы). Рекомендуется использовать **зашифрованные DNS-серверы** (например, 8.8.8.8, 1.1.1.1) или **VPN** для полной защиты.

2.02.3. Поток HTTPS (упрощённо)

Типичный процесс установки защищённого соединения:

1. Пользователь вводит `http://` (или просто домен). Браузер пытается соединиться по порту 80 (HTTP).
2. Сервер **перенаправляет** клиента на порт 443 (HTTPS) с кодом ответа 301 Moved Permanently.
3. Начинается **рукопожатие TLS** (TLS handshake):
 - Клиент отправляет **Client Hello** (информация о себе, поддерживаемые шифры).
 - Сервер отвечает **Server Hello** и отправляет свой **SSL-сертификат**.
 - Клиент проверяет сертификат, затем генерирует и отправляет свой ключ.
4. После подтверждения рукопожатия начинается **зашифрованная передача** данных (обычные HTTP-запросы/ответы, но уже внутри TLS).

Угроза: атака **понижения версии (downgrade attack)** — злоумышленник может попытаться принудительно переключить соединение на HTTP, чтобы перехватывать данные. Современные браузеры и серверы защищены от этого (HSTS, строгие политики).

2.02.4. cURL для HTTPS

Утилита **cURL** автоматически обрабатывает HTTPS: выполняет рукопожатие, шифрует и дешифрует данные. Однако, если сертификат сайта **недействителен** (просрочен, самоподписанный или не соответствует домену), cURL по умолчанию **откажется** выполнять запрос — это защита от MITM.

```
$ curl https://inlanefreight.com
curl: (60) SSL certificate problem: Invalid certificate chain
More details here: https://curl.haxx.se/docs/sslcerts.html
...SNIP...
```

Современные браузеры также предупреждают пользователя о проблемах с сертификатом.

Для тестирования локальных или учебных веб-приложений, у которых нет валидного сертификата, можно использовать флаг **-k** (или **-insecure**), чтобы пропустить проверку сертификата:

```
$ curl -k https://www.inlanefreight.com
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
...SNIP...
```

Теперь запрос успешно проходит, и мы получаем ответ.

2.02.42.1. Основные флаги cURL для работы с HTTPS

Флаг	Назначение
-k -insecure	“Игнорировать ошибки SSL-сертификата (использовать с осторожностью!)”
-cacert	“Указать собственный файл с корневыми сертификатами для проверки”
-cert	“Использовать клиентский сертификат для аутентификации”
-key	“Ключ для клиентского сертификата”

Подробнее: `man curl` или `curl -help all`.